JAVASCRIPT MASTERY SERIES · BOOK 3

JavaScript — DOM & Document

How JavaScript reads and changes the page: selecting elements, walking the document tree, and creating, updating, and removing content — taught vanilla, with notes on how frameworks like React do it differently.

10 TOPICS · 5 PARTS · BUILT FOR LEARNING & PRINTING

How to read this

Start at the **Navigation Tree** — it maps the whole language; every name jumps to its section.

Each topic carries a tier badge. CORE topics are taught in depth, connected to real web & data-app work — learn these cold. REFERENCE topics are mapped lightly: enough to recognize and look up. Inside a topic, exhaustive lists (every operator, every loop form) are always shown so nothing stays hidden.

Code is syntax-highlighted. Look for When to use notes — they explain *why* you'd reach for a construct and which alternative to pick. Version notes flag when a feature landed (ES2020+). A **API Index** at the end lists every operator and keyword.

Print or save as PDF with `Ctrl/Cmd + P` — the layout flows continuously and is print-tuned.

Navigation Tree

The whole language at a glance. teal = **core** (learn deeply) · grey = **reference** (know it exists). Every name links to its section.

Part 1 · Foundations

Before manipulating the page, you need a clear picture of what you’re actually manipulating: the DOM, the browser’s live model of the document.

What the DOM is CORE

IN PLAIN TERMS

When a browser loads an HTML page, it reads the HTML text and builds a living model of it in memory called the **DOM** (Document Object Model). The DOM is a **tree** of objects — one object per tag, piece of text, and comment — and JavaScript can read and change that tree. The crucial part: the DOM *is* what's on screen. Change the tree, and the page updates instantly. The original HTML file isn't touched — you're editing the live model the browser is showing.

Think of the HTML file as a blueprint read once at load. From then on, the browser shows the **DOM**, and your JavaScript works entirely with that. The whole tree hangs off one global object, `document`, which is your entry point for finding elements and making new ones.

Every item in the tree is a **node**. The ones you'll touch most are **element** nodes (from tags like `<div>`), but text and comments are nodes too. Elements nest to form the tree: a `<ul>` contains `<li>` children, which contain text. Understanding this parent/child structure is the foundation for everything else in this book.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`--`	Decrement operator — subtracts 1 from a number variable (`i--`).
`textContent`	textContent — a property holding an element's text with no HTML; safe from HTML injection.
`virtual DOM`	Virtual DOM — a framework's in-memory copy of the UI it diffs against the real DOM to apply minimal changes (React, Vue).
`declarative`	Declarative — describing *what* the UI should look like for given data and letting the framework update the DOM (vs. imperative step-by-step DOM calls).
`attribute`	Attribute — a name=value pair written in the HTML tag (`id="x"`, `href="..."`); the starting value of a property.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`property`	Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
`element`	Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.
`parent`	Parent — the node that directly contains another node.
`child`	Child — a node directly contained by another.
`event`	Event — a signal that something happened (a click, input, load); covered fully in the Events book.
`node`	Node — any single point in the DOM tree: an element, a piece of text, a comment, or the document itself.

`document` *is the root handle. Everything you select, read, or create starts from it.*

```
<!-- this HTML... -->
<body>
  <h1 id="title">Hello</h1>
  <ul class="list">
    <li>One</li>
    <li>Two</li>
  </ul>
</body>

/* ...becomes this DOM tree (simplified):
   document
   └─ html
      └─ body
         ├─ h1#title  → text "Hello"
         └─ ul.list
            ├─ li → text "One"
            └─ li → text "Two"
*/

// JavaScript reaches into that tree through `document`:
document.title;                       // the page title
document.body;                        // the <body> element
document.getElementById("title");     // the <h1>
document.querySelector(".list");      // the <ul>
```

IN REACT Frameworks like React, Vue, and Svelte mostly hide direct DOM manipulation. Instead of imperatively calling `document.createElement` and `append`, you write **declarative** markup (JSX/templates) describing what the UI should look like for the current data, and the framework figures out the minimal DOM changes — often via a **virtual DOM** (an in-memory copy it diffs). You still benefit from knowing the real DOM: it's what frameworks ultimately drive, it's what you debug, and you'll use these APIs directly in plain scripts, small widgets, and the inevitable cases where you reach outside the framework (focus management, measuring elements, third-party integrations).

IN ANGULAR Angular takes the declarative approach even further than React: you write HTML templates with bindings (`{{ value }}`, `[property]`, `(event)`) and Angular compiles them, keeping the DOM in sync with your component class. Rather than a virtual DOM, Angular uses change detection (and, in modern Angular, fine-grained **signals**) to know what to update. You rarely touch `document` directly — and when you must, Angular steers you toward its `Renderer2` abstraction (which can run outside the browser, e.g. server-side rendering) instead of raw DOM calls.

GOTCHA Your script can only touch DOM nodes that **exist when it runs**. A `<script>` in the `<head>` runs before the `<body>` is parsed, so `document.querySelector` finds nothing. Fix it by placing scripts at the end of `<body>`, adding the `defer` attribute, or waiting for the `DOMContentLoaded` event (covered in the Events book).

The core objects & node kinds

`document` `global`
The whole page as an object; the entry point for selecting and creating nodes. Properties like `document.body`, `document.title`.

`Node` `base type`

Any single point in the tree. Base type for elements, text, comments, and the document itself.

`Element` `node kind`

A node made from an HTML tag (`<div>`, `<a>`). What you select and manipulate most. `HTMLElement` is the common element subtype.

`Text node` `node kind`

The actual text inside an element. Often handled via `textContent` rather than touched directly.

`Comment node` `node kind`

An HTML comment in the tree. Rarely manipulated.

`DocumentFragment` `node kind`

An off-screen mini-tree for assembling nodes before inserting them efficiently (see Creating & inserting).

`window` `global`

The browser tab/window; `document` lives on it. Holds timers, navigation, and (later) events.

Part 2 · Selecting

Almost everything you do starts by finding the element(s) you want to work with. This part covers

searching the document with CSS selectors and moving between related nodes.

Finding elements CORE

IN PLAIN TERMS

To change something on the page, you first need a reference to it. **Selecting** means asking the document to find element(s) for you, usually with a **CSS selector** — the same patterns you use in stylesheets (`#id` , `.class` , `div > p`). The two methods you'll use for almost everything are `querySelector` (the first match) and `querySelectorAll` (all matches).

`document.querySelector(sel)` returns the **first** element matching a CSS selector, or `null` if none. `document.querySelectorAll(sel)` returns **all** matches as a `NodeList` you can loop over. These two cover virtually every case because they accept any CSS selector. The older `getElementById` / `getElementsByClassName` methods still work and are marginally faster, but `querySelector` is the modern default.

You can also call `querySelector` *on an element*, not just `document` , to search only within that element — essential for scoping a search to one component or section.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`?.`	Optional chaining operator — safely reaches into nested data; if a piece of the path is missing it stops and gives `undefined` instead of crashing.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`optional chaining`	Optional chaining — using `?.` to read nested data without crashing when part of it is missing.
`live collection`	Live collection — a node list that updates automatically as the DOM changes (e.g. getElementsByTagName).
`textContent`	textContent — a property holding an element's text with no HTML; safe from HTML injection.
`reference`	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`selector`	Selector — a CSS pattern (`#id` , `.class` , `div > p`) used to find elements in the page.
`spread`	Spread — using `...` to expand a list or object into its individual pieces.
`static`	Static member — a property or method that belongs to the class itself, not to individual instances.
`scope`	Scope — the region of code where a particular variable is visible and usable.
`child`	Child — a node directly contained by another.
`event`	Event — a signal that something happened (a click, input, load); covered fully in the Events book.

`querySelector` *takes the same selectors as CSS, so anything you can style you can select. Scope by calling it on an element to search just that subtree.*

```
// the two you'll use 95% of the time:
const title = document.querySelector("#title");      // first match (or null)
const items = document.querySelectorAll(".list li");  // ALL matches (NodeList)

// any CSS selector works:
document.querySelector("nav > a.active");
document.querySelector('input[name="email"]');
document.querySelector("ul li:first-child");

// loop over the results:
items.forEach(li ⇒ console.log(li.textContent));
for (const li of items) { /* ... */ }

// scope a search to within an element (not the whole document):
const list = document.querySelector(".list");
const firstItem = list.querySelector("li");          // searches inside `list` only

// older, still-valid, slightly faster for simple cases:
document.getElementById("title");                    // by id (no # prefix)
document.getElementsByClassName("list");             // LIVE collection
document.getElementsByTagName("li");                 // LIVE collection
```

GOTCHA `querySelector` returns `null` when nothing matches — calling a method on that null throws ("Cannot read properties of null"). Guard it: `const el = document.querySelector(...); if (el) { ... }` or use optional chaining `el?.textContent`. Also, the `NodeList` from `querySelectorAll` is a **static snapshot** — it doesn't update if the DOM changes, and it has `forEach` but not the full array methods (spread it with `[...list]` to use `map` / `filter`).

WHEN TO USE Selecting is the entry point to all DOM work — you can't change what you can't reference. `querySelector` / `querySelectorAll` won because they accept *any CSS selector*, so one mental model (CSS) covers both styling and scripting, and complex matches (`input[name="email"]:checked`) need no special code. **Scope to an element** (rather than always searching from `document`) when working inside a component or repeated section: it's faster, and it avoids accidentally grabbing a matching element elsewhere on the page. Reach for `getElementById` only in hot loops where its slight speed edge matters.

IN REACT In React you rarely select elements at all — instead of finding a node and reading it, the data already lives in component state, and you attach behavior in JSX. When you *do* need a raw DOM node (to focus an input, measure size, or integrate a non-React library), you use a **ref** (`useRef` + `ref={myRef}`) rather than `querySelector` , because querying the document can clash with the framework's own rendering. The selecting skills here still matter for debugging, plain scripts, and those escape hatches.

IN ANGULAR Angular discourages `document.querySelector` outright. To reach an element you use a **template reference variable** (`#myInput`) plus `@ViewChild('myInput')` in the component, which hands you an `ElementRef` after the view initializes (`ngAfterViewInit`). For finding several, `@ViewChildren` returns a `QueryList` . Querying the document directly fights Angular's view encapsulation and breaks server-side rendering, so the framework gives you these typed, lifecycle-aware accessors instead.

Selecting methods

`.querySelector()` `root.querySelector(sel)`
First element matching a CSS selector, or `null`. Call on `document` or any element. Your default.

`.querySelectorAll()` `root.querySelectorAll(sel)`
All matches as a **static** `NodeList`. Has `forEach`; spread for full array methods.

`.getElementById()` `document.getElementById(id)`
Element with that id (no `#`), or `null`. Fastest single lookup; document-only.

`.getElementsByClassName()` `root.getElementsByClassName(name)`
Live HTMLCollection of matches — updates as the DOM changes. No `forEach` (convert to array).

`.getElementsByTagName()` `root.getElementsByTagName(tag)`
Live collection of elements by tag name.

`.matches()` `el.matches(sel)`
Tests whether an element matches a selector — true/false. Handy in event handling and filtering.

`.closest()` `el.closest(sel)`
Walks **up** from an element to the nearest ancestor (or itself) matching a selector. Key for event delegation (Events book).

`document.activeElement` `property`
The element currently focused — useful for keyboard/focus work.

Traversing the tree CORE

IN PLAIN TERMS

Once you have one element, you often need a *related* one — its parent, its children, the next item in a list. **Traversing** means moving from a node to its neighbours in the tree. There are properties for going up (to the parent), down (to children), and sideways (to siblings).

Going **up**: `parentElement` and `closest(selector)` (the most useful — finds the nearest matching ancestor). Going **down**: `children` (element children) and `firstElementChild` / `lastElementChild`. Going **sideways**: `nextElementSibling` / `previousElementSibling`. Prefer the `…Element…` versions, which skip whitespace text nodes that otherwise trip people up.

▼ KEY TERMS IN THIS SECTION

`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`selector`	Selector — a CSS pattern (`#id`, `.class`, `div > p`) used to find elements in the page.
`traverse`	Traverse — to move through the DOM tree from one node to related nodes (parent, child, sibling).
`element`	Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.
`sibling`	Sibling — a node sharing the same parent as another node.
`dataset`	dataset — an element's `data-*` attributes exposed as a JavaScript object (`el.dataset.userId`).
`parent`	Parent — the node that directly contains another node.
`child`	Child — a node directly contained by another.
`event`	Event — a signal that something happened (a click, input, load); covered fully in the Events book.
`node`	Node — any single point in the DOM tree: an element, a piece of text, a comment, or the document itself.
`DOM`	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

`closest(selector)` *is the workhorse: from any element, jump up to the meaningful container (a row, a card, a form) in one call.*

```
const li = document.querySelector(".list li");

// up:
li.parentElement;                  // the <ul>
li.closest("ul");                  // nearest <ul> ancestor (or itself)
li.closest(".card");               // jump to an ancestor by selector

// down:
const ul = document.querySelector(".list");
ul.children;                       // HTMLCollection of the <li> elements
ul.firstElementChild;              // first <li>
ul.lastElementChild;               // last <li>
ul.childElementCount;              // how many element children

// sideways:
li.nextElementSibling;             // the next <li>
li.previousElementSibling;         // the previous <li> (or null)

// a real pattern — from a clicked button, find its row's data:
function onDelete(button) {
  const row = button.closest("tr");      // walk up to the table row
  const id = row.dataset.id;             // read its data-id
  row.remove();                          // and remove it
}
```

GOTCHA There are two parallel sets of traversal properties. The plain ones (`childNodes`, `firstChild`, `nextSibling`) include **text nodes** — and the whitespace between tags counts as text, so `firstChild` is often a confusing empty text node, not the element you expected. Use the `…Element…` versions (`children`, `firstElementChild`, `nextElementSibling`) which skip those. Reserve the node-level ones for when you specifically need text/comment nodes.

WHEN TO USE Traversal exists because elements are meaningful in relation to each other — a delete button belongs to a row, an input belongs to a form, a tab belongs to a tab group. Rather than giving every element a unique id and re-selecting from `document`, you start from the element you have (often the target of a click) and move to the related one. **`closest` is the standout**: it turns "this button is somewhere inside a card — give me the card" into one call, which is exactly the pattern behind event delegation (handle one listener on a container, then `closest` to find which item was acted on — detailed in the Events book).

IN REACT In framework code you almost never traverse the DOM manually — the component tree *is* your structure, and a child gets what it needs through **props**, not by reading `parentElement`. "Find the row this button belongs to" becomes "the row component passes its id to the button's handler." Manual traversal in React is a smell (and fragile, since the framework controls the DOM). It remains essential, though, in vanilla scripts and for event delegation patterns.

IN ANGULAR As in React, manual traversal is rare in Angular — structure lives in the component tree and data flows via `@Input()` / `@Output()` bindings, not by reading `parentElement`. When you genuinely need a related element you use `@ViewChild` / `@ContentChild` queries rather than walking the DOM. For the "which row did this button belong to?" pattern, Angular's template binds the row's data straight to the click handler (`(click)="onDelete(row)"`), so you already have the row object — no `closest` needed.

Going up

`.parentElement` `el.parentElement`
The element that contains this one (or `null`). Prefer over `parentNode`.

`.closest()` `el.closest(sel)`
Nearest ancestor (or self) matching a selector. The most useful traversal method.

Going down

`.children` `el.children`
Live HTMLCollection of **element** children (skips text nodes).

`.firstElementChild / .lastElementChild` `el.firstElementChild`
First / last element child.

`.childElementCount` `el.childElementCount`
Number of element children.

`.childNodes / .firstChild` `node-level`
Include text & comment nodes — usually not what you want (whitespace surprises).

`.querySelector() (scoped)` `el.querySelector(sel)`
Search *within* this element — often clearer than manual child-walking.

Going sideways

`.nextElementSibling` `el.nextElementSibling`
The next sibling element (or `null`).

`.previousElementSibling` `el.previousElementSibling`
The previous sibling element (or `null`).

`.nextSibling / .previousSibling` `node-level`
Include text/comment nodes — usually skip in favor of the Element versions.

Part 3 · Reading & Writing

With an element in hand, here’s how to read and change what it shows: its text and HTML content, its attributes and properties, and its classes and inline styles.

Content: text & HTML CORE

IN PLAIN TERMS

The most common change you'll make is updating what an element *says*. There are two main properties for this. `textContent` treats the content as plain text — safe and simple. `innerHTML` treats it as HTML — powerful (you can insert tags) but dangerous if the content comes from a user, because it can inject malicious code. The rule: use `textContent` unless you specifically need to insert HTML you trust.

`textContent` gets or sets the text inside an element, ignoring any HTML. `innerHTML` gets or sets the content as an HTML string, which the browser parses into real elements. For inserting a chunk of HTML at a specific position without replacing everything, `insertAdjacentHTML` is handy. Reading values: `textContent` for the text, `innerHTML` for the markup.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`interpolation`	Interpolation — inserting a value into the middle of a string (what `${ }` does).
`textContent`	textContent — a property holding an element's text with no HTML; safe from HTML injection.
`innerHTML`	innerHTML — a property holding an element's contents as an HTML string; setting it re-parses that HTML.
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`element`	Element — a node created from an HTML tag (a `<div>` , `<p>` , `<button>`); the most common kind of node.
`reflow`	Reflow — the browser recalculating element sizes/positions after a DOM or style change; expensive if done repeatedly.
`event`	Event — a signal that something happened (a click, input, load); covered fully in the Events book.
`XSS`	XSS (cross-site scripting) — an attack where malicious HTML/JS is injected via untrusted content; avoided by using textContent over innerHTML.

`textContent` *is your default. Reach for* `innerHTML` *only when you need to insert markup and you trust its source (your own template strings, not user input).*

```
const el = document.querySelector("#message");

// set plain text — safe, even with untrusted data:
el.textContent = "Hello, " + userName;      // any < > & shown literally, not parsed

// read the text:
const text = el.textContent;

// set HTML — only with content YOU control:
el.innerHTML = "<strong>Saved!</strong>";  // parsed into a real <strong> element

// DANGER — never do this with user input:
el.innerHTML = userComment;    // ✗ if userComment is "<img onerror=...>" → XSS attack

// insert HTML at a position without wiping existing content:
el.insertAdjacentHTML("beforeend", "<li>New item</li>");
// positions: "beforebegin" | "afterbegin" | "beforeend" | "afterend"
```

GOTCHA Setting `innerHTML` with untrusted content is the classic **XSS** (cross-site scripting) vulnerability — a user-supplied string like `<img src=x onerror="steal()">` executes as code. Always use `textContent` for user-provided values. Also, `el.innerHTML += "..."` re-parses and rebuilds the *entire* contents (slow, and it destroys existing event listeners and form state) — build the string fully then assign once, or use `insertAdjacentHTML` / `append`.

WHEN TO USE Two properties exist because there are two genuinely different needs: putting in *text* versus putting in *markup*. `textContent` is both faster and safe-by-default because the browser never interprets its value as HTML — perfect for the common case of showing data. `innerHTML` exists for when you legitimately need to create structure from a string (rendering a template), but it shifts responsibility to you to ensure the string is trusted. The security boundary between them is one of the most important things to internalize in front-end work: **data goes in via `textContent`; markup goes in via `innerHTML` only when you wrote it.**

IN REACT Frameworks make this safe by default: in React, `{userComment}` in JSX is automatically escaped (rendered as text), so XSS is prevented without you thinking about it — it behaves like `textContent`. Inserting raw HTML is deliberately made awkward and scary-named (`dangerouslySetInnerHTML`) precisely so you pause and confirm the source is trusted. That design is a direct response to how easy it is to misuse `innerHTML` in vanilla code.

IN ANGULAR Angular binds text safely with interpolation: `{{ userComment }}` is auto-escaped, like `textContent`. Inserting trusted HTML uses `[innerHTML]="html"`, and Angular runs it through its built-in **sanitizer** that strips dangerous content by default — to deliberately allow markup you must pass it through `DomSanitizer.bypassSecurityTrustHtml(...)`, whose scary name (like React's `dangerouslySetInnerHTML`) is meant to make you stop and confirm the source. So the same text-vs-HTML safety boundary from this section is enforced by the framework.

Content properties & methods

`.textContent` `el.textContent`
Get/set the element's text, ignoring HTML. Safe with untrusted data. Your default for showing values.

`.innerHTML` `el.innerHTML`
Get/set contents as an HTML **string** (parsed into elements). Powerful but XSS-risky — trusted content only.

`.insertAdjacentHTML()` `el.insertAdjacentHTML(pos, html)`
Insert an HTML string at a position (`beforeend`, etc.) without replacing existing content.

`.innerText` `el.innerText`
Like `textContent` but reflects rendered/visible text (respects CSS, triggers reflow). Usually prefer `textContent`.

`.outerHTML` `el.outerHTML`
The element *including* its own tag, as HTML. Setting it replaces the element entirely.

`.insertAdjacentText()` `el.insertAdjacentText(pos, text)`
Like `insertAdjacentHTML` but inserts plain text safely.

Attributes & properties CORE

IN PLAIN TERMS

HTML tags carry **attributes** — `id`, `href`, `src`, `disabled`, and so on. In JavaScript you can read and change these two ways: through methods like `getAttribute` / `setAttribute`, or through the element's **properties** (`el.id`, `el.href`). They usually mirror each other, but not always — a subtle distinction worth understanding. There's also a clean, dedicated way to store your own custom data: **`data-*` attributes**, read via `dataset`.

For standard attributes, the property form is easiest: `el.id`, `el.value`, `el.disabled`. Use `getAttribute` / `setAttribute` / `removeAttribute` for any attribute, especially non-standard ones. For your *own* data, use `data-*` attributes and read them through `el.dataset` — the proper place to stash things like a record id on an element. `hasAttribute` and `toggleAttribute` round it out.

▼ KEY TERMS IN THIS SECTION

`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`classList`	classList — an element property with methods to add/remove/toggle CSS classes.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`property`	Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
`element`	Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.
`dataset`	dataset — an element's `data-*` attributes exposed as a JavaScript object (`el.dataset.userId`).
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`DOM`	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

Use `dataset` for your own data — never invent non-standard attributes like `userid="42"`. `data-*` is the standard, future-proof place for it, and `dataset` camel-cases the names for you.

```
const link = document.querySelector("a");

// properties — easiest for standard attributes:
link.href;                        // full URL
link.id = "main-link";
const input = document.querySelector("input");
input.value = "hello";            // current value
input.disabled = true;            // boolean property

// attribute methods — work for ANY attribute:
link.getAttribute("href");        // the literal attribute string
link.setAttribute("target", "_blank");
link.removeAttribute("target");
link.hasAttribute("target");      // false
button.toggleAttribute("disabled");  // add if absent, remove if present

// custom data — the right way to attach your own values:
// <tr data-user-id="42" data-role="admin">
row.dataset.userId;               // "42"   (data-user-id → userId, camelCased)
row.dataset.role;                 // "admin"
row.dataset.userId = "99";        // sets data-user-id="99"
```

GOTCHA Attributes and properties can **diverge**. The attribute is the *initial* value from the HTML; the property is the *current* live value. For a text input, after the user types, `input.value` (property) reflects what they typed, but `input.getAttribute("value")` still shows the original HTML value. Similarly, the `href` property returns a *resolved absolute* URL while `getAttribute("href")` returns the literal (possibly relative) string. Rule of thumb: use **properties** for live state (form values, checked, disabled), **attributes** for the declared/initial config.

WHEN TO USE Both interfaces exist because of that attribute-vs-property split, which itself exists so the DOM can track *current* state separately from the *original* HTML. Properties (`el.value`, `el.disabled`) are the ergonomic path for the standard cases and give you live values with correct types (a boolean, a number) rather than always-strings. The attribute methods are the general escape hatch — they work for anything, including custom or ARIA attributes. `data-*` + `dataset` exists specifically to end the old bad habit of inventing made-up attributes: it's a sanctioned, collision-proof namespace for your own per-element data, which is exactly how you'll tag table rows with record ids in the data-table book later.

IN REACT In React you set these declaratively as JSX props — `<input value={x} disabled={isLocked} />` — and the framework syncs the DOM property for you; you don't call `setAttribute`. Custom data still uses `data-*` (`<tr data-user-id={id}>`), and React passes most unknown attributes straight through. One wrinkle React newcomers hit: a few names differ because they're JS properties, not HTML attributes — `class` becomes `className`, `for` becomes `htmlFor` — which makes more sense once you know the property-vs-attribute distinction from this section.

IN ANGULAR Angular makes the property-vs-attribute distinction explicit in its syntax. `[value]="x"` binds a DOM **property**; `[attr.aria-label]="x"` binds an **attribute** (note the `attr.` prefix) — exactly the difference this section describes. Booleans like disabled are `[disabled]="isLocked"`, and classes/styles have dedicated bindings (`[class.active]`, `[style.width.px]`). Custom data still uses `data-*` via `[attr.data-id]="id"`.

Properties (live, typed)

`.id / .value / .checked / .disabled` `el.prop`
Direct access to standard attributes as live, correctly-typed properties. Easiest for the common cases.

`.href / .src` `el.href`
Resolved (absolute) URL properties for links/media. Note these differ from the literal attribute.

`.className` `el.className`
The `class` attribute as a string. Usually prefer `classList` (next topic).

Attribute methods (any attribute)

`.getAttribute()` `el.getAttribute(name)`
The literal attribute string, or `null`. For non-standard/ARIA attributes or the declared value.

`.setAttribute()` `el.setAttribute(name, value)`
Set any attribute to a string value.

`.removeAttribute()` `el.removeAttribute(name)`
Remove an attribute entirely.

`.hasAttribute()` `el.hasAttribute(name)`
True/false — is the attribute present?

`.toggleAttribute()` `el.toggleAttribute(name, force?)`
Add if absent / remove if present (good for boolean attributes).

Custom data

`.dataset` `el.dataset.camelName`
Read/write `data-*` attributes as an object (`data-user-id` ↔ `dataset.userId`). The right home for your own per-element values.

Classes & styles CORE

IN PLAIN TERMS

How an element *looks* is controlled by CSS, and the cleanest way to change appearance from JavaScript is to add or remove **CSS classes** — letting your stylesheet define what each class does. The `classList` property gives you tidy methods for that (`add` , `remove` , `toggle`). You *can* set inline styles directly (`el.style.color = ...`), but reaching for a class is usually better.

`classList` is the tool: `add` , `remove` , `toggle` (flip on/off), and `contains` (check). This keeps styling decisions in your CSS where they belong — JavaScript just flips a class like `.is-active` or `.hidden` . The `style` property sets inline styles directly for the rare cases where a value is dynamic (a computed position, a progress width). `getComputedStyle` reads the *actual* applied style.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`--`	Decrement operator — subtracts 1 from a number variable (`i--`).
`declarative`	Declarative — describing *what* the UI should look like for given data and letting the framework update the DOM (vs. imperative step-by-step DOM calls).
`imperative`	Imperative — giving explicit step-by-step instructions (the direct DOM-manipulation style).
`attribute`	Attribute — a name=value pair written in the HTML tag (`id="x"` , `href="..."`); the starting value of a property.
`classList`	classList — an element property with methods to add/remove/toggle CSS classes.
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`property`	Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
`binding`	Binding — the link between a name and the value it refers to.
`element`	Element — a node created from an HTML tag (a `<div>` , `<p>` , `<button>`); the most common kind of node.
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`DOM`	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

Prefer `classList.toggle('.is-active')` over setting many inline styles: it keeps the look in CSS, supports states/themes, and is easier to maintain. Use `el.style` only for values that can't be known ahead of time.

```
const box = document.querySelector(".box");

// classList — the preferred way to change appearance:
box.classList.add("is-active");
box.classList.remove("hidden");
box.classList.toggle("open");           // flip on/off
box.classList.toggle("open", isOpen);   // force to a specific state
box.classList.contains("is-active");    // true/false
box.classList.replace("old", "new");

// inline styles — for genuinely dynamic values only:
box.style.width = progress + "%";       // values are STRINGS (with units!)
box.style.backgroundColor = "tomato";   // camelCase: background-color → backgroundColor
box.style.setProperty("--accent", "#11675e");   // CSS custom property

// read the ACTUAL applied style (not just inline):
const color = getComputedStyle(box).color;    // "rgb(...)" as the browser computes it
```

GOTCHA Inline style values are **strings and need units**: `el.style.width = 100` does nothing — it must be `"100px"` (or `"100%"`). Property names are camelCased (`backgroundColor`, not `background-color`). And `el.style.color` only reads back styles set *inline* — to get the real applied value from your stylesheet, use `getComputedStyle(el)`. Setting `el.style.cssText` or many properties in a row can trigger repeated **reflows**; batch changes (or toggle a single class) for performance.

WHEN TO USE Toggling classes exists as the preferred path because it preserves the **separation of concerns**: CSS decides how things look (including hover, theme, responsive variants), and JavaScript only decides *which state* an element is in by flipping a class. That's more maintainable than scattering colors and sizes through your JS, and it lets designers change appearance without touching code. Direct `style` manipulation exists for the genuinely dynamic cases a stylesheet can't express ahead of time — a drag position, a live progress bar width, a value from user input. `getComputedStyle` exists for *reading* the final result of all the CSS rules, which inline `style` alone can't tell you.

IN REACT Frameworks lean even harder on the class approach: you bind classes to state (`className={isActive ? "is-active" : ""}`, or helpers like `clsx`), so appearance is a declarative function of your data rather than imperative `add`/`remove` calls. Inline styles are supported (`style={{width: pct + "%"}}`) for dynamic values, and many apps use CSS-in-JS or utility frameworks (Tailwind) — but underneath, it's all still setting `className` and `style` on real DOM elements exactly as described here.

IN ANGULAR Angular has first-class bindings for exactly this: `[class.is-active]="isActive"` toggles a single class, `[ngClass]="{...}"` toggles several, and `[style.width.px]="w"` sets an inline style with a unit baked into the binding (no manual `+ 'px'`). It's the same "prefer classes, bind to state" philosophy from this section, expressed as template syntax — and component styles are scoped by Angular's view encapsulation so they don't leak.

classList (preferred)

`.classList.add() / .remove()` `el.classList.add(name)`
Add / remove one or more classes. The main way to change appearance.

`.classList.toggle()` `el.classList.toggle(name, force?)`
Flip a class on/off; pass a boolean to force a state. Great for show/hide, active states.

`.classList.contains()` `el.classList.contains(name)`
Check whether a class is present.

`.classList.replace()` `el.classList.replace(old, new)`
Swap one class for another.

`.className` `el.className`
The whole `class` attribute as a string. `classList` is usually cleaner.

Inline & computed styles

`.style.prop` `el.style.backgroundColor`
Set an inline style (camelCased, string values **with units**). For dynamic values only.

`.style.setProperty()` `el.style.setProperty(name, val)`
Set a style or CSS custom property (`--var`) by its real name.

`getComputedStyle()` `getComputedStyle(el)`
Read the **actual** applied styles (from all CSS), not just inline ones. Read-only.

`.style.cssText` `el.style.cssText`
Get/set all inline styles at once as a string.

Part 4 · Changing the Tree

Beyond editing existing elements, you’ll build new ones and restructure the page: creating nodes, inserting them in the right place, and removing, replacing, or moving what’s there.

Creating & inserting nodes CORE

IN PLAIN TERMS

To add something new to the page — a list item, a card, a row — you **create** an element in memory, fill it in, then **insert** it into the tree at the right spot. The modern methods (`append`, `prepend`, `before`, `after`) are flexible and accept text or multiple nodes at once. For adding many elements efficiently, you build them in a **DocumentFragment** (an off-screen container) and insert once.

`document.createElement(tag)` makes a new element; you set its content/attributes, then place it with `append` (inside, at the end), `prepend` (inside, at the start), or `before`/`after` (as a sibling). When adding many nodes in a loop, insert into a `DocumentFragment` first and append the fragment once — this avoids repeated layout work and is noticeably faster.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`concatenation`	Concatenation — joining pieces of text end to end into one string.
`textContent`	textContent — a property holding an element's text with no HTML; safe from HTML injection.
`virtual DOM`	Virtual DOM — a framework's in-memory copy of the UI it diffs against the real DOM to apply minimal changes (React, Vue).
`declarative`	Declarative — describing *what* the UI should look like for given data and letting the framework update the DOM (vs. imperative step-by-step DOM calls).
`reference`	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
`deep copy`	Deep copy — a fully independent duplicate, including nested objects (`structuredClone`).
`innerHTML`	innerHTML — a property holding an element's contents as an HTML string; setting it re-parses that HTML.
`classList`	classList — an element property with methods to add/remove/toggle CSS classes.
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`fragment`	DocumentFragment — a lightweight, off-screen container for building nodes before inserting them in one go.
`element`	Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.

Build in a `DocumentFragment` when inserting many nodes: each `append` to the fragment is cheap, and the single final insert into the page triggers just one layout pass instead of one per item.

```
// create, configure, insert:
const li = document.createElement("li");
li.textContent = "New item";
li.classList.add("item");
li.dataset.id = "42";

const list = document.querySelector(".list");
list.append(li);            // insert INSIDE, at the end
list.prepend(li);           // insert INSIDE, at the start
li.before(otherNode);       // insert as a sibling, before li
li.after(otherNode);        // insert as a sibling, after li

// append accepts text and multiple things at once:
list.append("plain text", anotherEl);

// efficient bulk insert — build off-screen, insert once:
const frag = document.createDocumentFragment();
for (const item of data) {
  const el = document.createElement("li");
  el.textContent = item.name;
  frag.append(el);          // appends to the fragment (no page reflow yet)
}
list.append(frag);          // ONE insertion into the live page

// clone an existing element as a template:
const copy = templateEl.cloneNode(true);   // true = deep copy (with children)
```

GOTCHA The old API (`appendChild` , `insertBefore`) still works but is clunmsier: `appendChild` takes only one node and returns it, while the modern `append` takes multiple nodes *and* strings and returns nothing. Don't confuse them. Also, inserting an element that's already in the document **moves** it (a node can only be in one place) — that's occasionally surprising but often useful. And building HTML by string concatenation + `innerHTML` is an XSS risk with untrusted data; `createElement` + `textContent` is safe.

WHEN TO USE Creating nodes programmatically is how any data-driven page is built — you fetch an array of records and turn each into an element. The modern insertion methods (`append` / `prepend` / `before` / `after`) exist to replace the awkward older ones with a single consistent set that accepts text and many nodes, reads naturally, and covers inside-vs-sibling placement. **The DocumentFragment matters for performance**: inserting elements one-by-one into the live page can force the browser to recalculate layout (a **reflow**) on every insert, which crawls with large lists; batching into a fragment makes it one reflow. This is the seed of the virtualization techniques in the data-table book — when even one reflow over thousands of rows is too slow, you render only what's visible.

IN REACT This is the single biggest thing frameworks take over. You never call `createElement` / `append` to render data — you write a declarative template (`data.map(item => <li>{item.name}</li>)`) and the framework creates, inserts, and later updates or removes the nodes for you, computing the minimal changes (often via a virtual DOM diff). It also handles the batching/efficiency that you'd manually use fragments for. You'll still use these APIs directly in vanilla scripts, Web Components, and when integrating non-framework libraries — and understanding them demystifies what React is doing under the hood.

IN ANGULAR Angular never has you call `createElement` / `append`. Structural directives generate and remove DOM for you: `@for` (or the older `*ngFor`) renders a node per item, `@if` / `*ngIf` adds/removes conditionally. Under the hood Angular manages the equivalent of fragments and batching. For the rare dynamic case it offers `ViewContainerRef` / `TemplateRef` and `Renderer2.createElement` — abstractions that work outside the browser too — rather than direct DOM construction.

Create

`document.createElement()` `document.createElement(tag)`
Make a new element (not yet in the page). Configure it, then insert.

`.cloneNode()` `node.cloneNode(deep)`
Copy a node; `true` includes its descendants. Useful for templating from an existing element.

`document.createTextNode()` `document.createTextNode(str)`
Make a standalone text node (usually `textContent` is simpler).

`document.createDocumentFragment()` `createDocumentFragment()`
An off-screen container for assembling many nodes before one efficient insert.

`<template>` `el.content.cloneNode(true)`
An HTML `<template>` element holds inert markup you clone to instantiate — a clean templating pattern.

Insert (modern)

`.append()` `parent.append(...nodes)`
Insert nodes/strings **inside**, at the end. Accepts multiple. Your default.

`.prepend()` `parent.prepend(...nodes)`
Insert inside, at the start.

`.before()` / `.after()` `el.before(...nodes)`
Insert as siblings, before/after this element.

`.insertAdjacentElement()` `el.insertAdjacentElement(pos, node)`
Insert one element at `beforebegin` / `afterbegin` / `beforeend` / `afterend`.

Insert (older, still valid)

`.appendChild()` `parent.appendChild(node)`
Insert one node at the end. Older; `append` is more flexible.

`.insertBefore()` `parent.insertBefore(node, ref)`
Insert one node before a reference child. `before` / `after` are usually clearer.

Removing, replacing & moving CORE

IN PLAIN TERMS

The flip side of adding: taking elements out, swapping one for another, or relocating them. The modern methods make this a one-liner — `el.remove()` deletes an element, `el.replaceWith(newEl)` swaps it, and `parent.replaceChildren(...)` clears and refills a container in one call.

`remove()` deletes an element from the tree. `replaceWith(...)` swaps an element for one or more others. `replaceChildren(...)` replaces *all* of a parent's children at once (call with no arguments to empty it cleanly). To move an element, just insert it elsewhere — inserting a node that's already in the document relocates it automatically.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`innerHTML`	innerHTML — a property holding an element's contents as an HTML string; setting it re-parses that HTML.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`closure`	Closure — a function that remembers and keeps using variables from where it was created.
`element`	Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.
`parent`	Parent — the node that directly contains another node.
`render`	Render — to produce or update what the user sees on screen from data.
`child`	Child — a node directly contained by another.
`event`	Event — a signal that something happened (a click, input, load); covered fully in the Events book.
`node`	Node — any single point in the DOM tree: an element, a piece of text, a comment, or the document itself.
`DOM`	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

`replaceChildren()` with no arguments is the modern, safe way to empty a container — clearer than `innerHTML = ""` and it doesn't re-parse anything.

```
const row = document.querySelector("tr.selected");

row.remove();                        // delete it from the page

row.replaceWith(newRow);             // swap it for another element
row.replaceWith(nodeA, nodeB);       // or several

// replace ALL children of a container (the clean way to re-render a list):
list.replaceChildren();              // empty it (better than innerHTML = "")
list.replaceChildren(...newItems);   // clear and refill in one operation

// "move" = just insert elsewhere (a node lives in one place only):
otherList.append(row);               // removes from old parent, adds to new

// older equivalents:
parent.removeChild(row);             // returns the removed node
parent.replaceChild(newRow, row);
```

GOTCHA Emptying a container with `el.innerHTML = ""` works but re-parses an empty string and silently discards any event listeners and form state inside — `el.replaceChildren()` is cleaner and clearer. Also, removing an element doesn't remove references to it: if your code still holds the variable (or a closure does), the node stays in memory until those references are gone — a subtle leak source in long-running apps. Detached event listeners on removed nodes are usually cleaned up, but lingering references are not.

WHEN TO USE These exist to express structural edits directly. Before them, removing an element meant `el.parentNode.removeChild(el)` (you had to reach up to the parent just to delete a child); `el.remove()` says what you mean. `replaceChildren` solves the extremely common "clear this list and put new items in" — the heart of re-rendering — in one call, replacing the old `while (el.firstChild) el.removeChild(el.firstChild)` loop or the leaky `innerHTML = ""`. The move-by-inserting behavior (a node can only exist in one location, so inserting relocates) is *why* you don't need a separate "move" method — and it's handy for reordering lists.

IN REACT Removal and reordering are fully automated in frameworks: when an item leaves your data array, the framework removes its DOM node; when the array reorders, it moves nodes — driven by **keys** that let it track which item is which (`data.map(x => <li key={x.id}>…</li>)`). Getting keys right is how React efficiently moves rather than rebuilds. Manual `remove` / `replaceChildren` is for vanilla code; in a framework, you change the data and let it reconcile the DOM.

IN ANGULAR Like React's keys, Angular's `@for` requires a `track` expression (`@for (item of items; track item.id)`) so it can tell which DOM node maps to which item and **move** nodes instead of rebuilding them when the list reorders. Removing an item from your array removes its node automatically. So removal/reordering is data-driven here too; manual `remove()` / `replaceChildren()` is only for the rare escape-hatch case via `Renderer2`.

Remove / replace

`.remove()` `el.remove()`
Delete the element from the tree. The clean one-liner.

`.replaceWith()` `el.replaceWith(...nodes)`
Swap this element for one or more nodes/strings.

`.replaceChildren()` `parent.replaceChildren(...nodes)`
Replace all children at once; no args empties the container. The re-render workhorse.

`.removeChild()` `parent.removeChild(child)`
Older removal; returns the removed node. `remove()` is simpler.

`.replaceChild()` `parent.replaceChild(new, old)`
Older swap of one child for another.

Move

`(insert elsewhere)` `newParent.append(el)`
Inserting a node already in the document **moves** it — no separate move method needed.

Part 5 · Elements in Depth

A closer look at the object model behind elements, and how to measure their size, position, and visibility — the readouts you need for layout-aware behavior.

The node & element hierarchy REFERENCE

IN PLAIN TERMS

Everything in the DOM is built from a family of related object types. Knowing the rough hierarchy explains *why* a `<div>` has methods a text node doesn't, and where properties like `textContent` or `querySelector` come from. You don't need to memorize it — just understand the layering so the available methods make sense.

At the base is `EventTarget` (anything that can receive events). `Node` builds on it (anything in the tree — elements, text, comments). `Element` builds on `Node` (tag-based nodes, adding `classList`, `querySelector`, attributes). `HTMLElement` adds HTML-specifics (`style`, `dataset`), and specific tags have their own types (`HTMLInputElement` adds `value`, `HTMLAnchorElement` adds `href`). Each layer inherits everything below it.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`textContent`	textContent — a property holding an element's text with no HTML; safe from HTML injection.
`prototype`	Prototype — the object a JavaScript object inherits shared properties and methods from.
`classList`	classList — an element property with methods to add/remove/toggle CSS classes.
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`dataset`	dataset — an element's `data-*` attributes exposed as a JavaScript object (`el.dataset.userId`).
`DOM`	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

This is why every element has `addEventListener`, `textContent`, and `classList`: they're inherited from `EventTarget`, `Node`, and `Element` respectively. A specific type like `HTMLInputElement` just adds its own extras on top.

```
/* the inheritance chain (general → specific):

   EventTarget          can receive events (addEventListener)
     └ Node             is in the tree (textContent, childNodes, parentNode)
       └ Element        from a tag (classList, querySelector, getAttribute)
         └ HTMLElement   HTML element (style, dataset, click())
           └ HTMLInputElement   adds .value, .checked
           └ HTMLAnchorElement  adds .href
           └ ...one type per tag
*/

const input = document.querySelector("input");
input instanceof HTMLInputElement;   // true
input instanceof HTMLElement;        // true (inherits)
input instanceof Element;            // true
input instanceof Node;               // true

input.value;        // from HTMLInputElement
input.classList;    // from Element
input.textContent;  // from Node
input.addEventListener; // from EventTarget
```

WHEN TO USE This layering exists so shared behavior is defined once and inherited, rather than repeated on every element type. `addEventListener` lives on `EventTarget` so *everything* that can receive events gets it; `querySelector` lives on `Element` so every element can search within itself; `value` lives only on form-control types because only they have a value. Understanding the chain answers practical questions — "why does my `<div>` not have a `.value`?" (it's not a form control), "can I call `closest` on this?" (yes, if it's an `Element`) — and it's the same prototype-inheritance idea from Book 1, applied by the browser to model the page.

The hierarchy (general → specific)

`EventTarget` base
Anything that can receive events. Source of `addEventListener` (Events book).

`Node` base
Anything in the tree. Source of `textContent`, `childNodes`, `parentNode`, `appendChild`.

`Element` tag node
Tag-based nodes. Source of `classList`, `querySelector`, attributes, `closest`, `getBoundingClientRect`.

`HTMLElement` HTML element
HTML-specific element. Source of `style`, `dataset`, `hidden`, `click()`, `focus()`.

`HTMLInputElement`, `HTMLAnchorElement`, … per-tag types
Tag-specific types adding their own properties (`value`, `href`, `src`, etc.).

`instanceof` check
Test an element's type: `el instanceof HTMLInputElement`. Useful for narrowing before using type-specific properties.

Size, position & visibility CORE

IN PLAIN TERMS

Sometimes you need to know *where* an element is or *how big* it is — to position a tooltip next to it, detect if it's scrolled into view, or measure available space. The DOM exposes these as read-only measurements. The main one, `getBoundingClientRect()`, returns an element's size and position relative to the viewport (the visible window).

`getBoundingClientRect()` is the key method — it returns `{top, left, right, bottom, width, height}` relative to the viewport. For scroll position there's `scrollTop` / `scrollLeft` and `window.scrollY`. For raw dimensions, `offsetWidth` / `offsetHeight` (including borders) and `clientWidth` / `clientHeight` (inside padding). To act when an element enters the screen, the modern, efficient tool is `IntersectionObserver`.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`**`	Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
`imperative`	Imperative — giving explicit step-by-step instructions (the direct DOM-manipulation style).
`document`	document — the global object representing the whole page; the entry point for selecting and creating nodes.
`viewport`	Viewport — the visible area of the page in the browser window.
`pending`	Pending — a promise's starting state, before it has succeeded or failed.
`element`	Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.
`module`	Module — a single code file with its own scope that shares code via `import` / `export`.
`render`	Render — to produce or update what the user sees on screen from data.
`reflow`	Reflow — the browser recalculating element sizes/positions after a DOM or style change; expensive if done repeatedly.
`event`	Event — a signal that something happened (a click, input, load); covered fully in the Events book.
`node`	Node — any single point in the DOM tree: an element, a piece of text, a comment, or the document itself.
`DOM`	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

`getBoundingClientRect()` is viewport-relative, so add `window.scrollY`/`scrollX` to get document-relative coordinates. For "is it visible?" logic, prefer `IntersectionObserver` over manually comparing rects on scroll.

```
const box = document.querySelector(".box");

// size + position relative to the viewport:
const rect = box.getBoundingClientRect();
rect.top; rect.left; rect.width; rect.height;   // all in pixels
// position a tooltip just below the box:
tooltip.style.top = (rect.bottom + window.scrollY) + "px";

// dimensions:
box.offsetWidth;    // full width incl. padding + border
box.clientWidth;    // width inside padding (no border/scrollbar)
box.scrollHeight;   // full content height, even if overflowing

// scroll position:
window.scrollY;     // how far the page is scrolled down
box.scrollTop;      // how far THIS element's content is scrolled

// scroll an element into view:
box.scrollIntoView({behavior: "smooth", block: "center"});

// efficiently detect when an element enters the screen (lazy-load, infinite scroll):
const io = new IntersectionObserver((entries) => {
  for (const e of entries) if (e.isIntersecting) load(e.target);
});
io.observe(box);
```

GOTCHA Reading layout properties (`getBoundingClientRect`, `offsetWidth`, etc.) **forces a synchronous reflow** — the browser must finish pending layout to answer. Interleaving reads and writes in a loop (read width, set width, read width, …) causes "layout thrashing" and janky performance. Batch your reads together, then your writes. Also, these measure the *rendered* element — a `display:none` element reports zeros, and the values are affected by CSS transforms.

WHEN TO USE These readouts exist for behavior that depends on actual rendered geometry, which CSS alone can't express: positioning a popover relative to its trigger, drag-and-drop, detecting scroll position, measuring before/after animations. `getBoundingClientRect` is the one-stop measurement because it returns everything (size and viewport position) consistently. `IntersectionObserver` exists specifically to fix a performance problem: the old way to detect "is this on screen?" was to recompute rects on every scroll event, which was slow and janky; the observer lets the browser report visibility changes efficiently, off the main work — which is exactly what powers lazy-loading images and infinite-scroll, techniques you'll use in the data-table book for rendering large data.

IN REACT Frameworks don't replace these — measuring the real DOM is inherently imperative — so you reach for them through escape hatches: a **ref** to get the node, then `getBoundingClientRect` inside an effect (`useEffect`/`useLayoutEffect`) that runs after render. Libraries like Floating UI (popovers) and virtualization libraries wrap exactly these APIs plus `IntersectionObserver`. So even in a framework app, this is the layer you drop to whenever behavior depends on real size or position.

IN ANGULAR Measuring the real DOM is imperative everywhere, so Angular routes you through its abstractions: get the node via `@ViewChild(...)` → `ElementRef.nativeElement`, then call `getBoundingClientRect()` inside `ngAfterViewInit`. Angular's CDK (Component Dev Kit) wraps the heavier lifting — `cdkScrollable`, the `Overlay` package (built on these measurements for popovers/tooltips), and a virtual-scroll module that uses the same viewport math as `IntersectionObserver`. So this section's APIs are exactly what Angular's layout tooling sits on top of.

Measure size & position

`.getBoundingClientRect()` `el.getBoundingClientRect()`
`{top,left,right,bottom,width,height}` relative to the viewport. The main measurement tool. Forces reflow — batch reads.

`.offsetWidth / .offsetHeight` `el.offsetWidth`
Rendered size including padding and border (integer pixels).

`.clientWidth / .clientHeight` `el.clientWidth`
Inner size: content + padding, excluding border and scrollbar.

`.scrollWidth / .scrollHeight` `el.scrollHeight`
Full content size including overflow that isn't visible.

Scroll

`window.scrollY / scrollX` `global`
How far the page is scrolled. Add to `getBoundingClientRect` for document coordinates.

`.scrollTop / .scrollLeft` `el.scrollTop`
How far an individual scrollable element's content is scrolled (settable).

`.scrollIntoView()` `el.scrollIntoView(opts)`
Scroll the element into the visible area; `{behavior:"smooth"}` animates.

Observe

`IntersectionObserver` `new IntersectionObserver(cb)`
Efficiently react when an element enters/leaves the viewport — lazy-load, infinite scroll. Preferred over scroll-handler math.

`ResizeObserver` `new ResizeObserver(cb)`
React when an element's size changes — responsive components, charts.

`.hidden / .checkVisibility()` `el.hidden`
`hidden` toggles display; `checkVisibility()` reports whether an element is actually rendered/visible.

API Index

Every operator, keyword, and statement form in one place — the quick lookup for “what does this symbol mean?”

TOKEN	KIND	MEANING
`.querySelector()`	Select	First element matching a CSS selector, or null.

TOKEN	KIND	MEANING
`.querySelectorAll()`	Select	All matches as a static NodeList.
`.getElementById()`	Select	Element by id (no #). Fastest single lookup.
`.getElementsByClassName()`	Select	Live collection of elements by class.
`.getElementsByTagName()`	Select	Live collection of elements by tag.
`.matches()`	Select	Does this element match a selector? true/false.
`.closest()`	Traverse	Nearest ancestor (or self) matching a selector.
`.parentElement`	Traverse	The containing element.
`.children`	Traverse	Live collection of element children (skips text).
`.firstElementChild` / `.lastElementChild`	Traverse	First / last element child.
`.nextElementSibling` / `.previousElementSibling`	Traverse	Adjacent sibling elements.
`.textContent`	Content	Get/set text, ignoring HTML. Safe default.
`.innerHTML`	Content	Get/set contents as an HTML string. XSS-risky.
`.insertAdjacentHTML()`	Content	Insert an HTML string at a position without replacing content.
`.outerHTML`	Content	The element including its own tag, as HTML.
`.getAttribute()` / `.setAttribute()`	Attribute	Read / write any attribute (string).
`.removeAttribute()` / `.hasAttribute()`	Attribute	Remove / check an attribute.
`.dataset`	Attribute	Read/write data-* attributes as an object.
`.id` / `.value` / `.checked` / `.disabled`	Property	Live, typed standard properties.
`.classList.add()` / `.remove()` / `.toggle()`	Class	Add / remove / flip CSS classes. Preferred for appearance.
`.classList.contains()`	Class	Is a class present?
`.style.prop`	Style	Set an inline style (camelCase, string values with units).
`getComputedStyle()`	Style	Read the actual applied styles (read-only).
`document.createElement()`	Create	Make a new element.
`.cloneNode()`	Create	Copy a node (deep with true).
`createDocumentFragment()`	Create	Off-screen container for efficient bulk insert.
`.append()` / `.prepend()`	Insert	Insert nodes/strings inside (end / start).

TOKEN	KIND	MEANING
`.before() / .after()`	Insert	Insert as siblings.
`.appendChild() / .insertBefore()`	Insert	Older single-node insertion.
`.remove()`	Remove	Delete the element from the tree.
`.replaceWith()`	Replace	Swap an element for one or more nodes.
`.replaceChildren()`	Replace	Replace all children (no args empties). Re-render workhorse.
`(insert elsewhere)`	Move	Inserting an in-document node moves it.
`EventTarget / Node / Element / HTMLElement`	Hierarchy	The inheritance chain elements are built from.
`instanceof`	Hierarchy	Test an element's specific type.
`.getBoundingClientRect()`	Geometry	Size + viewport position. Forces reflow — batch reads.
`.offsetWidth / .offsetHeight`	Geometry	Rendered size incl. padding + border.
`.clientWidth / .clientHeight`	Geometry	Inner size: content + padding.
`window.scrollY / scrollX`	Scroll	Page scroll position.
`.scrollIntoView()`	Scroll	Scroll an element into view (smooth optional).
`IntersectionObserver`	Observe	Efficiently detect entering/leaving the viewport.
`ResizeObserver`	Observe	React to element size changes.